

Low-Level Design (LLD) Practice Platform

Execution Guide, System Architecture, Limitations & AI Usage Report

> **CipherSchools Engineering Hiring Assignment — September 2026** > **Repository:**

<https://github.com/sachinn-alt/llid-studio> > **Deliverable:** Combined README & AI Usage Report

PART I: README & SYSTEM DOCUMENTATION

1. The Learner Problem

Unlike Data Structures & Algorithms (where LeetCode evaluates correctness through deterministic binary test cases), **Low-Level Design (LLD) is notoriously difficult to evaluate.**

Two valid designs for the same problem (e.g. Parking Lot or Elevator System) can look completely different depending on scale and trade-offs.* * Learners fall into the "**passive reading trap**"—reading pre-baked UML diagrams gives an illusion of mastery, but when designing from scratch, learners have no way to verify whether their classes violate Single Responsibility, have leaky abstractions, or introduce tight coupling. *Unconstrained LLMs (e.g., asking ChatGPT "Is this a good design?") hallucinate arbitrary scores without citing concrete evidence or maintaining consistent standards.*

The Solution

This platform solves **one clear learner problem exceptionally well**: 1. Provides **5 curated, constraint-rich LLD problems** (Multi-Floor Parking Lot, Multi-Car Elevator Dispatcher, Splitwise Expense Sharing, In-Memory Cache with Pluggable Eviction, Distributed API Rate Limiter). 2. Requires **structured design articulation** (Entities & Responsibilities, Interfaces & Contracts, Design Patterns with Justifications, Trade-offs & Concurrency). 3. Bridges design to code via **Live Interactive UML Diagrams** and **Multi-Language Boilerplate Export** (Java, TypeScript, C++). 4. Evaluates solutions through a **two-stage hybrid engine**: fast deterministic checks followed by a semantic rubric evaluation citing **concrete evidence, design smells/concerns**, and **actionable refactorings**. 5. Tracks **attempt history and progression deltas** ($A_1 \rightarrow A_2 \rightarrow A_3$) to foster real, measurable design improvement.

2. Repository & Deliverables Layout

```
``  lld-practice-platform/ |— docs/ | |— RESEARCH_NOTE.md # 2-page deep dive on learner problem
& market gaps | |— RESEARCH_NOTE.pdf # Publication-grade PDF for Google Form submission | |—
DESIGN_NOTE.md # Domain architecture, UML diagrams, patterns, trade-offs | |— DESIGN_NOTE.pdf #
Publication-grade PDF for Google Form submission | |— README_AI_USAGE.md # Consolidated
documentation & AI report | |— README_AI_USAGE.pdf # Publication-grade PDF for Google Form
submission |— src/ | |— api/ | |— routes.ts # REST API endpoints (Problems, Attempts,
```

```

Submissions, History) | └─ data/ | | └─ problems.json # 5 Curated LLD problems with rubrics &
starter templates | └─ domain/ | | └─ entities/ | | | └─ Attempt.ts # Finite state machine
(DRAFT -> SUBMITTED -> EVALUATING -> COMPLETED) | | | └─ Feedback.ts # Multi-dimensional rubric
feedback with evidence citations | | | └─ Problem.ts # Problem specifications & domain
constraints | | | └─ Rubric.ts # Criteria dimensions, weights, and score definitions | | | └─
StructuredDesignPayload.ts # Strongly typed structured submission model | | | └─ Submission.ts #
Generic submission wrapper (Proves Change Test A) | | | └─ interfaces/ | | | └─
IAttemptRepository.ts | | | └─ IProblemRepository.ts | | | └─ ISubmissionPayload.ts # Core
submission abstraction | | └─ value-objects/ | | | └─ AttemptStatus.ts # DRAFT
SUBMITTED EVALUATING COMPLETED FAILED | | └─ SubmissionFormat.ts # STRUCTURED_TEXT UML_DIAGRAM
CODE | | └─ ValidationResult.ts | └─ evaluators/ | | | └─ IEvaluator.ts # Strategy contract
(Proves Change Test B) | | | └─ DeterministicRuleEvaluator.ts # Fast structural & schema validator
| | | └─ SemanticRubricEvaluator.ts # Evidence-grounded rubric engine (Live LLM + Local fallback)
| | └─ CompositeEvaluator.ts # Composite Pattern coordinating evaluation pipeline | └─
infrastructure/ | | └─ repositories/ # In-memory repositories with domain interfaces | └─
services/ | | └─ PracticeService.ts # Application service orchestrating attempt lifecycles | └─
server.ts # Backend entry point | └─ frontend/ # Vite + React practice studio UI | └─ src/ | |
└─ components/ | | | └─ AttemptHistoryView.tsx # Multi-attempt comparison & score delta | | |
└─ EvaluationFeedbackView.tsx # Rubric dashboard with evidence & suggestions | | | └─
Navbar.tsx | | | └─ ProblemCatalog.tsx # Problem browser | | | └─ ProblemWorkspace.tsx #
Interactive structured design studio (Live UML & Code Exporter) | | | └─ App.tsx | | └─ index.css
# Modern dark glassmorphism styling | └─ tests/ | | └─ domain/ | | | └─ Attempt.test.ts # Lifecycle
state machine & transition rules | | └─ SubmissionPayload.test.ts # Change Test A (UML diagram
format proof) | └─ evaluators/ | | └─ CompositeEvaluator.test.ts # Change Test B (Pluggable
human reviewer proof) | └─ services/ | └─ PracticeService.test.ts # End-to-end attempt
creation, submission & retry delta | └─ scripts/ | └─ generate-pdfs.js # Automated headless PDF
compiler for markdown docs | └─ AI_USAGE.md # 4 documented AI-assisted architectural decisions |
└─ README.md # Root execution guide

```

3. Quick Start (Running Locally)

Prerequisites

* **Node.js:** v18.0.0 or higher (v22.x tested) * **npm:** v9.0.0 or higher

Step 1: Install Dependencies

```
` bash
```

Install backend and development dependencies

```
npm install
```

Install frontend dependencies

```
cd frontend && npm install && cd ..`
```

Step 2: Run Automated Tests

```
`bash npm test` (All 12 automated unit and integration tests will execute and pass via Vitest).
```

Step 3: Run Full Build

```
`bash npm run build` (Runs TypeScript compiler tsc for backend and Vite production build for frontend).
```

Step 4: Start Local Development Servers

```
`bash`
```

Run both backend API and frontend studio concurrently:

```
npm run dev` * Backend API: http://localhost:3001 * Frontend Practice Studio: http://localhost:5173`
```

4. Key Architectural Highlights & LLD Principles

4.1 Finite State Machine for Attempt Lifecycle

Submissions are never processed as a blocking, synchronous HTTP call. The `Attempt` entity strictly enforces legal state transitions: `DRAFT` $\rightarrow$ `SUBMITTED` $\rightarrow$ `EVALUATING` $\rightarrow$ `COMPLETED` $\vee$ (`or` `FAILED`)

* **Crash Resilience:** The submission payload is persisted immediately in the database as `SUBMITTED`. If the evaluation service times out or crashes, the learner's work is never lost and can be retried with one click. * **Idempotency:** Repeated button clicks while evaluating are safely rejected.

4.2 The Two Change Tests (Extensibility)

Change Test A: New Submission Formats (e.g. UML Diagrams or Code)

* **Design Pattern:** Polymorphic Payload Abstraction (`ISubmissionPayload`). * **Proof:** We can introduce `UmlDiagramPayload` or `CodeSubmissionPayload` implementing `ISubmissionPayload`. The `Attempt` entity, `PracticeService`, and storage layer require **zero modifications** (verified in tests/domain/SubmissionPayload.test.ts).

Change Test B: New Evaluators (e.g. Human Review or Static AST Linter)

* **Design Pattern:** Strategy Pattern (`IEvaluator`) and Composite Pattern (`CompositeEvaluator`). * **Proof:** To add peer human review, we simply create `HumanReviewEvaluator` implements `IEvaluator`

and register it in `new CompositeEvaluator([...])`. The practice orchestration flow does not change by a single line (verified in `tests/evaluators/CompositeEvaluator.test.ts`).

4.3 Practical Scale Roadmap

* The current MVP runs as a clean, modular monolith. * When scaling to 50,000 active users, the **first and only component to decouple** is the **Evaluation Worker**: 1. The API saves the attempt and publishes an event to an AWS SQS queue or Redis Stream. 2. Stateless evaluation workers consume from the queue and run `CompositeEvaluator`. 3. The domain core (`Attempt` , `Problem` , `Rubric` , `Feedback`) remains identical.

5. Scope Limitations & Engineering Trade-offs

1. **In-Memory Storage**: Domain state is managed in-memory via `InMemoryProblemRepository` and `InMemoryAttemptRepository` implementing repository interfaces (`IProblemRepository` , `IAttemptRepository`). This ensures zero-setup execution for reviewers while maintaining dependency inversion to swap for PostgreSQL/DynamoDB seamlessly. 2. **Local Hybrid Evaluation**: Evaluation runs 100% reliably on local machines using the deterministic rule evaluator and semantic rubric matcher without requiring paid third-party API keys. 3. **No Peripheral Bloat**: We deliberately avoided multi-tenant auth, social feeds, and payment gateways to keep 100% focus on deep object-oriented domain modeling and the learner feedback loop.

PART II: AI USAGE REPORT

Assignment Requirement: Section 6 (AI Usage - 3 to 5 meaningful AI-assisted decisions) **Author**: Candidate **Date**: September 2026

Overview

In developing this Low-Level Design Practice Platform, AI tools (Claude, Gemini, ChatGPT) were used as an architectural sounding board and pair programmer. In every interaction, AI suggestions were subjected to rigorous engineering evaluation against the core project objective: **solving one clear learner problem exceptionally well without unnecessary complexity**.

Below are 4 meaningful architectural decisions highlighting what the AI proposed, what was accepted or rejected, and the engineering rationale.

Decision 1: System Topology – Microservices vs. Modular Monolith

* **What the AI Suggested**: The AI initially suggested a distributed microservices topology consisting of 4 independent services: an `AuthService` , `ProblemService` , `SubmissionService` , and an asynchronous `EvaluationService` communicating via an Apache Kafka message broker with Redis caching. * **Our Evaluation**: The suggestion was a classic over-engineering trap. For a 2-day prototype focusing on LLD domain modeling, introducing Kafka, Docker Compose orchestration,

distributed transactions, and network serialization adds operational fragility without adding any value to the learner's feedback loop. * **Decision: REJECTED.** * **Engineering Rationale:** We implemented a **clean, modular monolith** in TypeScript. Domain boundaries are enforced at the module and package level rather than through network boundaries. Domain events run via an in-memory event emitter. We documented a practical scale roadmap showing that the *Evaluation Worker Pool* is the only component that would be extracted if traffic scaled, keeping the MVP zero-config and lightning-fast.

Decision 2: Evaluation Design – Single-Prompt Scoring vs. Hybrid Rubric Engine

* **What the AI Suggested:** The AI suggested a simple, all-in-one prompt: *"Analyze the candidate's design and assign an overall score from 1 to 100 with general suggestions for improvement."* * **Our Evaluation:** This approach produces vague, uncalibrated feedback. An LLM without strict constraints exhibits high variance (scoring the same design 65 on one run and 85 on another), praises trivial naming conventions, and provides generic platitudes (*"Consider using more design patterns"*). Furthermore, it wastes expensive tokens evaluating malformed or incomplete submissions. * **Decision: REJECTED the single prompt; ACCEPTED a Two-Stage Hybrid Rubric Engine.** * **Engineering Rationale:** We split evaluation into two decoupled stages: 1. **Deterministic Rule Evaluator:** Instant (<50ms) checks for schema validity, mandatory sections, entity count, and minimum description depth. Fails fast without wasting LLM resources. 2. **Semantic Rubric Evaluator:** The LLM is strictly constrained to a fixed JSON schema with orthogonal dimensions (*Single Responsibility, Coupling & Abstraction, Requirement Coverage, Trade-offs*). Crucially, the model is required to cite **specific evidence** (class/method names) from the candidate's text alongside every identified concern.

Decision 3: Submission Extensibility (Change Test A) – Strongly Typed Generic Payload

* **What the AI Suggested:** When designing the `Submission` class, the AI provided a concrete class containing hardcoded string fields:

```
`typescript // AI Initial Suggestion
class Submission {
  entitiesText: string;
  relationshipsText: string;
  patternsText: string;
}
```

 * **Our Evaluation:** While simple, this hardcoding violates the **Open-Closed Principle (OCP)**. When evaluating the brief's **Change Test A** (*"Today the learner submits text; later the platform supports a class diagram"*), this structure would force breaking changes across the database schema, domain classes, and evaluation services. * **Decision: ACCEPTED refactoring to an `ISubmissionPayload` abstraction.** * **Engineering Rationale:** We refactored `Submission` to encapsulate an `ISubmissionPayload` interface:

```
`typescript export interface ISubmissionPayload {
  readonly format: SubmissionFormat;
  validate(): ValidationResult;
  toEvaluationContext(): string;
}

export class Submission {
  constructor(
    public readonly id: SubmissionId,
    public readonly payload: T,
    public readonly submittedAt: Date
  ) {}
}
```

 Now, supporting `UmlDiagramPayload` or `CodeSubmissionPayload` requires zero modifications to `Attempt` or `PracticeService`.

Decision 4: Submission State Lifecycle – Synchronous Request vs. Stateful Asynchronous Transition

*** What the AI Suggested:** The AI proposed a standard synchronous REST endpoint: `typescript`
`app.post('/api/attempts/:id/submit', async (req, res) => { const feedback = await`
`evaluator.evaluate(req.body); res.json({ status: 'COMPLETED', feedback }); });` *** Our**
Evaluation: LLM evaluations take between 2 to 5 seconds depending on model latency. A synchronous HTTP request creates multiple failure modes: 1. If the user's connection drops or the browser tab closes during the 4-second wait, the candidate's work is lost forever. 2. If the LLM provider times out or throttles, the entire request fails catastrophically. 3. Clicking "Submit" twice triggers parallel duplicate evaluations. *** Decision: REJECTED synchronous flow; ACCEPTED stateful asynchronous lifecycle.** *** Engineering Rationale:** We introduced an explicit state machine on the Attempt entity: `$$\text{DRAFT} \rightarrow \text{SUBMITTED} \rightarrow \text{EVALUATING} \rightarrow \text{COMPLETED} \mid (\text{or } \text{FAILED})$$` When a candidate clicks submit: 1. The submission is persisted immediately, and status transitions to SUBMITTED. 2. The API responds with 202 Accepted within 10ms. 3. Evaluation executes asynchronously. If evaluation fails, the state becomes FAILED with a retry option, ensuring candidate work is never lost.